

Estructura de Datos - Hoja de Cálculo Web

1. Introducción

Para representar la hoja de cálculo internamente y garantizar su eficiencia, se he elegido una estructura de datos basada en una **Tabla Hash plana**, donde la clave principal será el identificador de la celda (ej. 'A1', 'B2'). Siendo la principal la razón de esta selección, que el diseño garantiza un acceso directo $O(1)$, facilita la vinculación con las listas de adyacencia y la eficiencia en la memoria, a diferencia de una matriz bidimensional tradicional que desperdicia espacio en celdas vacías.

Con este diseño, la primera clave de la tabla hash será la fila (ej. 1, 2, 3), y su valor apuntará a una segunda tabla hash donde las claves son las columnas (ej. A, B, C). Dentro de esta segunda tabla se encuentra el objeto final de la celda. Como alternativa funcional, también se puede almacenar en una tabla hash plana donde la clave es el nombre directo (ej. A1, B2).

Dentro de cada nombre de celda, el valor almacenado contendrá toda la información relevante, agrupada dentro de un objeto:

- **El valor en texto ingresado:** La fórmula o el texto original ingresado por el usuario.
- **El valor calculado:** El resultado matemático final.
- **El árbol de sintaxis abstracta (AST):** La estructura que representa en orden jerárquico las operaciones de la formula ingresada.
- **Dependencias:** Las referencias hacia las celdas de las cuales necesita valores para operar (de las que depende) y las celdas a las cuales afecta si esta celda cambia de valor (las que dependen de ella).

2. Transición de PSeInt a Código (Objetos vs. Matrices)

En las etapas diseño en PSeInt, al no existir un manejo nativo de objetos, se utilizó el esquema de tablas o matrices para representar tanto el árbol AST como las relaciones de las celdas. Por ejemplo, en PSeInt los nodos del árbol se guardaban en las filas de **matrizAST**, ingresando todos los datos secuencialmente por posiciones y números de fila.

Sin embargo, a la hora de pasar el programa a la codificación real, nosotros vamos a utilizar **Objetos** para poder manipular de una forma mucho más fácil y escalable los datos. Esta

transición permitirá estructurar en formato JSON todo lo que en PSeInt requería complejas matrices multidimensionales.

Ejemplo: Estructura de una Celda (JSON)

Este es un ejemplo de cómo se generaría el objeto para representar la estructura principal en base a la lógica de Tabla Hash de Objetos:

```
{
  "hojaCalculo": {
    "A1": {
      "textoOriginal": "10",
      "valorCalculado": 10,
      "arbolAST": null,
      "dependencias": [],
      "dependientes": ["B1"]
    },
    "B1": {
      "textoOriginal": "=A1 + 5",
      "valorCalculado": 15,
      "arbolAST": {
        "tipo": "Operador",
        "valor": "+",
        "nodoIzquierdo": { "tipo": "Celda", "valor": "A1" },
        "nodoDerecho": { "tipo": "Numero", "valor": 5 }
      },
      "dependencias": ["A1"],
      "dependientes": ["C1"]
    },
    "C1": {
      "textoOriginal": "=B1 * 2",
      "valorCalculado": 30,
      "arbolAST": {
        "tipo": "Operador",
        "valor": "*",
        "nodoIzquierdo": { "tipo": "Celda", "valor": "B1" },
        "nodoDerecho": { "tipo": "Numero", "valor": 2 }
      },
      "dependencias": ["B1"],
      "dependientes": []
    }
  }
}
```

3. Relaciones y Dependencias entre Celdas

Para gestionar adecuadamente cómo el cambio en una celda afecta a otras en el recálculo en cadena, se utilizó la estructura de datos de **Grafos Dirigidos**. Este tipo de grafo es el ideal ya que solo nos interesa saber qué celdas proporcionan información hacia la celda padre de manera unidireccional, y así sucesivamente con las celdas hijas.

El modelo en PSeInt (Matriz de Adyacencia): Como PSeInt no maneja objetos, para poder representar esta estructura de grafos se utilizaba una matriz de adyacencia (denominada `matrizDependencias`). En ella, se asignaba un “1” en las intersecciones de los índices numéricos para representar la conexión o dependencia entre el origen y el destino.

El modelo en Código Real (Lista de Adyacencia): A la hora de codificar vamos a utilizar una lista de adyacencia, debido a que es una forma mucho más eficiente cuando las relaciones entre los nodos son dispersas. Se va a guardar por medio de un objeto o un arreglo de identificadores esas listas, que optimizarán muchísimo la velocidad de recorrido y el uso de memoria en comparación a la matriz tradicional.

Ejemplo: Si la celda C1 (Índice 3) depende de A1 (Índice 1) y B1 (Índice 2):

Modelo PSeInt (Matriz de Adyacencia):

```
matrizDependencias[3, 1] <- 1  
matrizDependencias[3, 2] <- 1
```

Modelo en Código (Lista de Adyacencia en JSON):

```
{  
  "A1": { "dependientes": ["C1"] },  
  "B1": { "dependientes": ["C1"] },  
  "C1": { "dependeDe": ["A1", "B1"] }  
}
```

4. Estructura del Árbol de Sintaxis Abstracta (AST)

El AST es fundamental para desglosar y resolver fórmulas matemáticamente y jerárquicamente, identificando valores izquierdos, derechos y el tipo de operación.

Modelo PSeInt (Representación Tabular en Matriz):

El AST dependía de almacenar los tipos, operadores, valores y enlaces (por índice de fila) a través de la matriz de 4 columnas llamada matrizAST.

Ejemplo:

```
matrizAST[1, 1] <- "Celda"
matrizAST[1, 2] <- "B1"

matrizAST[2, 1] <- "Numero"
matrizAST[2, 2] <- "10"

matrizAST[3, 1] <- "Operador"
matrizAST[3, 2] <- "+"
matrizAST[3, 3] <- "1" // Apunta al hijo izquierdo
matrizAST[3, 4] <- "2" // Apunta al hijo derecho
```

Modelo en Código (Objeto Anidado / Estructura Jerárquica):

En el código se usará una estructura jerárquica de objetos dentro de objetos sin depender de posiciones físicas en arreglos. Esto permitirá que el evaluador recorra el árbol AST utilizando recursividad matemática, haciendo el código mucho más limpio y eficiente.

```
{
  "tipo": "Operador",
  "valor": "+",
  "nodoIzquierdo": { "tipo": "Celda", "valor": "B1" },
  "nodoDerecho": { "tipo": "Numero", "valor": 10 }
}
```